学号: 202209518

上海立达学院
SHANGHAI LIDA UNIVERSITY

本科毕业论文（设计）

中国金融期货大数据分析与风险评估系统设计与实现

学院名称 ： 数字科学学院

专业名称 ：数据科学与大数据技术

学生姓名 ： 郭子健

指导教师 ： 马利平

2026 年 3 月 20 日

BACHELOR'S DEGREE THESIS OF

SHANGHAI LIDA UNIVERSITY

Design and Implementation of a Big Data Analytics and Risk Assessment System for Chinese Financial Futures

Candidate: Guo Zijian

Supervisor: Ma Liping

Mar. 20th, 2026

郑重声明

本人郑重声明：所呈交的论文是本人在导师的指导下独立进行研究所取得的研究成果。除了文中特别加以标注引用的内容外，本论文不包含任何其他个人或集体已经发表或撰写的成果作品。对本文的研究做出重要贡献的个人和集体，均已在文中以明确方式标明。本人完全意识到本声明的法律后果由本人承担。

毕业论文（设计）成果归上海立达学院所有。

作者专业：**数据科学与大数据技术**

作者学号：202109423

作者签名：

年　月　日（手填时间）

中国金融期货大数据分析与风险评估系统设计与实现

摘要

在金融市场波动加剧和股指期货风险管理需求持续上升的背景下，利用大数据技术对金融期货行情进行持续监测、结构化存储和量化分析具有现实意义。针对公开行情来源分散、字段格式不统一、盘后分析链路割裂等问题，本文以中国金融期货交易所 IF（沪深 300）、IH（上证 50）、IC（中证 500）和 IM（中证 1000）四类股指期货为研究对象，设计并实现中国金融期货大数据分析与风险评估系统。

系统存储层采用 DuckDB 与 Parquet 结合的轻量数据仓方案，开发层采用 FastAPI 异步接口框架和 Streamlit 前端展示技术。核心功能包括官方行情采集、主力连续序列构建、EWMA 波动率预测、历史模拟法风险价值（Value at Risk，VaR）计算、宏观相关性分析和数据可视化。实验结果表明，系统已采集 2025 年 12 月至 2026 年 3 月的真实行情数据，形成 912 条期货日频记录和 228 条主力连续记录；在 95% 置信水平、1 天持有期和 60 日回溯窗口下能够输出 VaR 结果。API 功能验证与压力测试均可正常返回，主要分析接口 50 次连续请求的 95 分位响应时间低于 220 ms，DuckDB 百万级模拟扫描聚合平均耗时约 0.0047 s，能够支持多维度可视化分析。

关键词: 金融期货；大数据分析；波动率预测；风险价值（VaR）；DuckDB；数据可视化

Design and Implementation of a Big Data Analytics and Risk Assessment System for Chinese Financial Futures

ABSTRACT

Against the background of increasing financial market volatility, continuous monitoring and risk assessment of financial futures are important for financial engineering practice. To solve the problems of scattered public data sources, inconsistent structures, and fragmented post-market analysis workflows, this thesis designs and implements a big data analytics and risk assessment system for Chinese financial futures, covering IF (CSI 300), IH (SSE 50), IC (CSI 500), and IM (CSI 1000) stock index futures listed on the China Financial Futures Exchange.

At the storage layer, DuckDB and Parquet support high-performance local analytical queries. At the development layer, FastAPI provides asynchronous analytical APIs, while Streamlit builds the visualization front end. The core functions include official data collection, main contract construction, EWMA volatility forecasting, historical simulation Value at Risk (VaR) calculation, macro-factor correlation analysis, and data visualization. The system collected real market data from December 2025 to March 2026, including 912 daily futures records and 228 main-contract records. With a 95% confidence level, a one-day holding period, and a 60-day lookback window, it can generate VaR results. API stress testing shows that the 95th percentile response time of major analytical interfaces is below 220 ms, and a DuckDB simulated million-row aggregation test took about 0.0047 seconds on average.

Key words: financial futures; big data analytics; volatility forecasting; Value at Risk (VaR); DuckDB; data visualization

目　录

1 绪论.....1

1.1 研究背景与意义.....1

1.2 国内外研究现状.....1

1.3 研究内容与研究方法.....3

1.4 论文结构安排.....3

2 相关技术与理论基础.....4

2.1 金融期货盘后分析的数据特征.....4

2.2 系统实现相关技术.....4

2.3 波动率与风险度量理论.....5

2.4 本文方案的适用性说明.....6

3 需求分析与数据资源设计.....7

3.1 系统建设目标.....7

3.2 功能需求分析.....7

3.3 非功能需求分析.....8

3.4 数据源设计.....8

3.5 数据模型设计.....9

4 系统总体设计.....10

4.1 总体架构设计.....10

4.2 模块划分设计.....11

4.3 核心数据表设计.....11

4.4 API 与页面设计.....12

5 系统详细实现与关键技术.....13

5.1 数据采集实现.....13

5.2 主力连续序列构建.....14

5.3 波动率预测与风险评估实现.....15

5.4 服务层实现.....16

5.5 前端展示实现.....17

5.6 本章小结.....20

6 系统测试与结果分析.....21

6.1 测试环境与测试策略.....21

6.2 功能测试结果.....21

6.3 数据规模与质量分析.....22

6.4 分析结果展示.....23

6.5 测试与结果分析总结.....26

7 结论与展望.....27

7.1 研究结论.....27

7.2 项目价值与不足.....27

7.3 后续改进方向.....28

参考文献.....29

致谢.....31

1 绪论

1.1 研究背景与意义

金融期货是现代金融市场中重要的风险管理工具，具有价格发现、套期保值和资产配置等多重功能。伴随我国资本市场规模扩张和金融衍生品体系逐步完善股指期货在服务实体经济、稳定预期和提升市场效率方面的作用日益突出。与此同时，市场参与者对于盘后数据分析的要求不断提高，不仅希望获得收盘价格、成交量和持仓量等基础信息，更需要在统一框架下观察主力合约迁移、收益波动特征、风险暴露水平及其与宏观变量之间的联动关系。

传统的期货数据分析往往存在三个突出问题。其一，数据来源分散。交易所日统计、宏观金融统计和宏观经济指标分别发布于不同官方站点，获取与整理成本较高。其二，分析链路割裂。很多课程项目只关注爬取或可视化中的某一个环节，缺少从原始数据到分析结果、再到页面展示的完整闭环。其三，可复用性不足。开发成果通常停留在脚本级别，无法稳定形成论文图表、实验截图和答辩展示页面，导致技术实现与论文撰写之间存在明显断层。

从研究意义上看，构建中国金融期货大数据分析与风险评估系统具有明显的现实价值和教学价值。一方面，系统化整合期货行情和宏观指标，有助于提升盘后风险分析与趋势研判的效率；另一方面，采用真实公开数据而非人工构造数据可以增强毕业设计作品的可信度与可验证性。对于数据科学与大数据技术专业的本科毕业设计而言，这样的课题既能体现数据采集、数据仓、统计分析和可视化展示等综合能力，又能形成具有完整交付形态的软件原型与学术论文成果。

1.2 国内外研究现状

国内外关于股指期货市场波动、风险与交易影响的研究已形成较为系统的积累。陈其安等围绕投资者结构和股指期货交易对中国股票市场波动性的影响进行了理论与实证分析，指出股指期货交易制度与投资者结构的变化会对市场稳定性产生显著作用[1]。Bohl 等则基于中国股票市场研究了指数期货交易对波动率的影

响，强调不同市场结构下波动传导路径存在差异[2]。这些研究表明，围绕股指期货开展波动和风险分析具有明确的理论基础和现实需求。

在方法层面，波动率建模与风险度量是金融时间序列分析中的核心问题。Engle 提出的 ARCH 模型为刻画条件异方差提供了经典框架[3]，Bollerslev 进一步提出 GARCH 模型，用更灵活的滞后结构描述波动持续性[4]。RiskMetrics 技术文档提出了 EWMA 波动率估计方法[5]，Kupiec 则从风险模型检验视角提出了针对风险测度准确性的验证思路[6]。这些方法广泛应用于资产收益率波动分析和风险管理实践，为本文系统中的波动率预测和 VaR 计算提供了直接的方法来源。

在工程实现方面，近年的研究和产业实践越来越强调数据处理与分析平台的一体化。DuckDB 的嵌入式分析数据库方案在中小规模分析场景下具有部署简单、查询高效的特点[7]；Apache Spark 为分布式数据处理提供了成熟框架[8]；FastAPI 和 Streamlit 等轻量框架则降低了分析服务和可视化前端搭建门槛[9][10]。与传统“脚本 + 报表”的离散式做法相比，将采集、存储、分析、服务与展示集成在统一系统中，更能满足教学展示和论文写作的双重需求。

总体而言，现有研究在理论方法方面已经较为丰富，但在本科毕业设计语境下，仍然存在“理论强而系统弱”或“系统有界面但缺乏真实数据链路”的常见问题。基于此，本文选择以真实公开数据为核心，以可重复的工程实现为支撑，构建一套兼顾学术表达与系统落地的中国金融期货交易分析系统。

近年研究进一步体现出金融期货分析与大数据方法融合的趋势。Chen 和 Hu 针对中美股指期货波动率预测构建混合 LSTM 模型，说明深度学习方法可用于处理不规则金融时间序列[15]；Lu 等从波动率预测视角检验中国期货市场对现货市场的引导作用，为跨市场风险监测提供了实证证据[16]；Zeng 等利用中国期货市场波动信息预测行业股票市场波动，表明期货数据能够为风险预警提供增量信息[17]。上述研究为本文选择股指期货行情、波动率预测和 VaR 风险评估作为系统核心功能提供了参考。

在学位论文框架方面，近三年相关硕士论文普遍采用“数据来源说明、模型方法设计、实验验证、系统或策略应用”的写作逻辑。例如，何婧萱围绕机器学习与 A 股市场股指期货价格预测展开研究[20]，朱峰围绕金融市场高频交易数据波动率预测开展深度学习应用研究[21]。本文在借鉴其框架逻辑的基础上，更强调从真实公开数据采集、轻量数据仓建设到可视化系统交付的工程闭环。

1.3 研究内容与研究方法

本文的研究内容主要包括以下四个方面。第一，围绕中国金融期货市场盘后分析场景，完成系统需求分析，明确数据采集、行情展示、趋势分析、风险度量和质量监控等核心功能。第二，设计轻量数据仓与系统总体架构，实现期货行情宏观指标和交易所通知的统一管理。第三，完成主力连续构建、滚动波动率计算未来五日波动率预测、宏观相关性分析和历史模拟法 VaR 计算等关键分析功能。第四，基于学校论文模板完成论文图表、代码片段和系统截图的组织与输出，形成可复现的毕业设计交付流程。

在研究方法上，本文综合采用文献研究法、系统分析法、原型开发法和实证分析法。文献研究法用于梳理股指期货波动分析、风险测度和大数据平台实现等理论基础；系统分析法用于抽象业务需求和模块边界；原型开发法用于循序推进采集、分析和展示模块的落地；实证分析法则依赖真实公开数据，对系统功能和结果进行验证，并以 2026 年 3 月 19 日的最新快照作为阶段性分析样本。

1.4 论文结构安排

本文共分为七章。第一章为绪论，介绍研究背景、研究意义、研究现状、研究内容与论文结构。第二章介绍系统实现所依赖的关键技术与理论基础。第三章对系统需求和数据资源设计进行分析。第四章阐述系统总体架构、数据流和模块设计。第五章对数据采集、数据仓建设、分析算法和前端展示的实现进行详细说明。第六章给出系统测试过程与分析结果。第七章总结全文并提出后续改进方向。

2 相关技术与理论基础

2.1 金融期货盘后分析的数据特征

与股票日频分析相比，金融期货数据具有合约到期、主力切换、持仓量敏感和宏观联动性强等特点。单个合约在生命周期内的成交与持仓水平会随到期日临近发生明显变化，因此若直接使用固定合约观察长期走势，容易引入合约切换导致的序列断裂问题。为此，系统需要先对同一品种的多个月份合约进行筛选，构建能够反映市场主流交易重心的主力连续序列。

此外，金融期货作为杠杆交易工具，波动率和风险暴露对投资决策具有更强约束作用。单纯展示价格曲线难以满足分析需求，还应同时提供收益率、回撤、滚动波动率和 VaR 等指标。考虑到期货市场与宏观环境之间存在联动关系，将货币供应量、社会融资规模、CPI、PPI 等指标纳入同一数据仓，有助于在盘后分析中提升解释性。

从统计特性看，金融期货收益率序列通常并不完全服从正态分布，而是容易呈现波动聚集、尖峰厚尾和尾部风险集中的特征。所谓尖峰厚尾，是指大多数交易日收益率集中在均值附近，但极端涨跌出现的概率高于正态分布假设。该特性使得仅使用均值和方差难以充分描述下行风险，因此本文在波动率预测之外引入历史模拟法 VaR，用经验分位数刻画给定置信水平下的潜在损失。

2.2 系统实现相关技术

本文系统的底层数据处理以 Python 3.11 为基础，使用 Pandas 完成表格数据整理，使用 DuckDB 和 Parquet 完成数据落盘和查询。DuckDB 作为嵌入式分析数据库，面向 OLAP 场景提供列式执行、向量化计算和本地进程内查询能力，省去了独立数据库服务部署成本，能够较好支持本科毕业设计场景下的本地分析与接口服务[8][18][19]。Parquet 则为列式存储格式，适合保存结构化分析结果，便于后续图表导出与 Spark 兼容扩展。

与传统 MySQL 更偏向事务型增删改查和在线业务系统不同，本文系统的主要任务是对日频行情、主力连续、波动率和风险指标进行批量聚合、分组统计与多维查询，属于典型 OLAP 分析场景。DuckDB 可直接读取本地 Parquet 文件

并在单机环境下完成向量化聚合，避免了 MySQL 需要额外建库、建索引和维护服务进程的复杂度，因此更适合本系统“轻量部署、快速分析、便于复现”的设计目标。

在服务层，系统采用 FastAPI 暴露 REST 风格分析接口。FastAPI 具有类型注解友好、接口定义清晰、自动文档生成等优点，适合将分析层计算结果封装为可复用 API[9]。在展示层，系统采用 Streamlit 构建交互式页面，以较低工程复杂度实现指标卡片、行情图、风险图和数据源说明页，有利于答辩展示和论文截图生成[10]。对于扩展性需求，项目保留了 Spark local mode 的演示接口，以说明后续面向更大规模数据处理时的演进方向[8]。

2.3 波动率与风险度量理论

收益率是金融时间序列分析的基础。设第 t 日收盘价为 P_t，则简单收益率定义如下。

$$r_t = \frac{P_t - P_{t-1}}{P_{t-1}} \quad (2\text{-}1)$$

其中，r_t 表示第 t 日收益率，P_t 和 P_{t-1} 分别表示相邻两个交易日的收盘价。基于收益率序列，可以进一步定义滚动窗口下的年化波动率。

$$\sigma_t = \sqrt{252} \times Std\left(r_{t-n+1}, \ldots, r_t\right) \quad (2\text{-}2)$$

式中，n 为滚动窗口长度，Std 表示样本标准差，252 表示一年内近似交易日数量。为提升对近期市场变化的敏感度，系统同时引入 EWMA 方式估计条件波动率。

$$\sigma_t^2 = \lambda \sigma_{t-1}^2 + (1-\lambda) r_t^2 \quad (2\text{-}3)$$

其中，λ 为衰减系数，本文实现中采用 0.94 作为经验参数。风险价值 VaR 用于刻画给定置信水平下的潜在最大损失，本文采用历史模拟法，其定义如下。

$$VaR_{c,L} = -Q_{(1-c)}\left(r_{t-L+1}, \ldots, r_t\right) \quad (2\text{-}4)$$

其中，c 为置信水平，L 为回溯窗口，Q 为经验分位数函数。考虑到市场表现与宏观变量之间的联动性，系统还计算收益率与宏观指标之间的 Pearson 相关系数。

$$\rho_{xy} = \frac{\sum_{i=1}^{n}}{\sqrt{\left(\sum_{i=1}^{n}(x_i-\bar{x})^2\right)\left(\sum_{i=1}^{n}(y_i-\bar{y})^2\right)}} \quad (2\text{-}5)$$

上述公式共同构成了系统分析层的理论基础。其中，收益率与滚动波动率用于描述市场短期变化，EWMA 用于增强近期波动信息的权重，历史模拟法 VaR

用于给出风险阈值，相关系数用于观察期货与宏观变量之间的联动关系。相较于复杂深度学习模型，这套方法更适合毕业设计阶段强调可解释性、可验证性和工程可控性的系统实现。

2.4 本文方案的适用性说明

开题报告中提出了基于大数据框架和机器学习的扩展设想，这一方向具有合理性，但在毕业设计当前交付范围内，最重要的是保证系统具备真实数据来源、清晰工程结构和稳定运行能力。基于此，本文采取“轻量数据仓 + 经典金融分析方法 + 快速可视化展示”的方案。该方案既能够保留未来向分布式处理和更复杂预测模型扩展的空间，又能够确保当前论文中的每个模块都有明确代码、明确数据和明确结果支撑。

3 需求分析与数据资源设计

3.1 系统建设目标

结合开题报告与实际项目实现，本文系统的建设目标可以概括为三个层面。第一，完成对中国金融期货公开数据的统一采集和结构化管理，解决多源数据分散的问题。第二，实现趋势、波动率、相关性和 VaR 等核心分析功能，形成可解释的盘后分析结果。第三，提供便于课程展示和论文取证的界面输出，使系统页面、图表和数据表能够直接服务于毕业设计文档。

3.2 功能需求分析

表 3.1 系统功能需求汇总

功能模块	具体需求	对应实现
数据采集	采集 CFFEX 日统计、交易所通知、PBC 金融统计报告、NBS 宏观数据发布	`ingest/` 模块
数据存储	保留原始网页和 XML，建设统一数仓	`data/raw`、DuckDB、Parquet
行情分析	查询期货日行情、主力连续走势、成交量与持仓量	`main_contract_daily`、市场页面
趋势分析	计算均线、累计收益、回撤与滚动波动率	`analysis/metrics.py`
风险分析	计算波动率预测、历史模拟 VaR、相关性矩阵	波动率预测与风险页面
可视化展示	页面化展示分析结果并输出论文截图	`app/streamlit_app.py`
数据质量	统计数据规模、缺失率、抓取成功率	`quality_report`

从功能上看，系统并不面向实盘自动交易，而是定位于盘后分析平台。这一定位决定了系统应重点强调数据真实性、指标解释性和结果展示完整性，而不是追求毫秒级交易时延。该定位与学校毕业设计要求更为匹配，也符合公开数据授权边界。

3.3 非功能需求分析

表 3.2 系统非功能需求汇总

非功能需求	具体要求	实现方式
真实性	数据必须来自官方公开渠道，能够追溯源地址	记录 `source_url` 与 `fetch_time`
稳定性	本地环境可重复运行，主要流程不依赖复杂集群	`DuckDB + Parquet`、本地脚本
可维护性	模块边界清晰，便于扩展数据源和分析方法	按 `ingest`、`analysis`、`api`、`app` 分层
可展示性	页面结构清晰，支持论文截图与答辩展示	Streamlit 页面与截图脚本
可复核性	图表、快照、测试结果能够与代码和数据库对应	导出图表脚本与测试脚本

3.4 数据源设计

系统采用三类官方公开数据源，分别承担行情、宏观金融和宏观经济三个层面的信息供给。

表 3.3 官方数据源及采集内容

数据源	主要内容	在系统中的作用
中国金融期货交易所	四类股指期货日统计、交易所通知	构建行情数据、主力连续、通知摘要
中国人民银行调查统计司	M2、M1、贷款同比、社融存量、同业拆借利率	作为宏观金融因子参与相关性分析
国家统计局数据发布	CPI、PPI、工业增加值等	作为宏观经济因子参与相关性分析

截至 2026 年 3 月 20 日，系统已入仓 912 条期货日频记录、18 条宏观指标记录和 20 条交易所通知记录。其中，四个品种各拥有 228 条日频行情记录，各 57 条主力连续记录；宏观指标包含 m2_yoy、m1_yoy、loan_yoy、interbank_rate、social_financing_stock_yoy、cpi_yoy、ppi_yoy 和 industrial_output_yoy 等序列。这些数据规模虽然尚不足以支持极长周期统计研究，但足以支撑毕业设计场景下的系统功能验证与结果展示。

3.5 数据模型设计

系统采用“原始层、标准层、分析层”三层数据设计。原始层保存网页、HTML 和 XML 文件，以保证数据可追溯；标准层统一字段命名和数据类型，形成 contracts、futures_daily、macro_series、notice_events 等基础表；分析层基于标准层继续生成 main_contract_daily、analysis_snapshot、volatility_forecast、correlation_matrix 和 quality_report 等结果表。

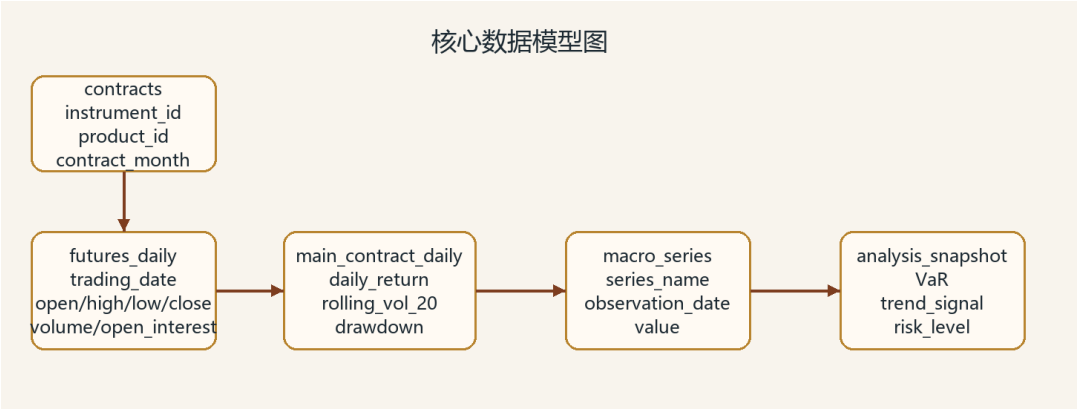

图 3.1 核心数据模型图

这种数据模型的优势在于既能支持逐表查询和接口返回，也能直接服务于论文中的表格和图表输出。对于毕业设计而言，这种兼顾分析与交付的模型设计比仅保存单个 CSV 文件更具工程意义。

4 系统总体设计

4.1 总体架构设计

系统总体上采用“采集层、存储层、分析层、服务层、展示层”五层结构。采集层负责访问官方站点并提取结构化字段；存储层负责原始数据落盘和 DuckDB/Parquet 持久化；分析层负责主力连续、波动率预测、VaR 和相关性等指标计算；服务层通过 FastAPI 对外提供统一接口；展示层通过 Streamlit 进行可视化交互与论文截图输出。该结构既保证了模块之间的低耦合，也便于后续替换数据源和扩展分析方法。

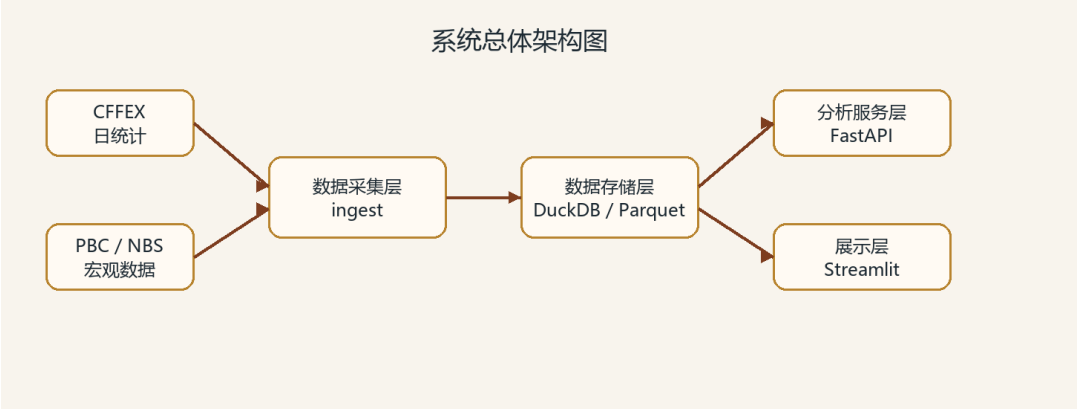

图 4.1 系统总体架构图

从数据流角度看，系统按照“原始行情获取 -> DuckDB/Parquet 存储 -> 算法模型计算 -> FastAPI 接口输出 -> Streamlit 页面渲染”的流程运行。CFFEX 日统计与通知、PBC 报告和 NBS 数据发布首先进入采集层，然后以原始文件形式写入 data/raw/；经过解析与标准化后，被写入 DuckDB 与 Parquet；分析层读取数仓表后生成主力连续、EWMA 波动率预测、历史模拟 VaR、快照和质量报告等结果表，再由 API 与前端页面进行消费。这一链路实现了“原始数据可追溯、分析结果可复用、页面展示可重建”的闭环。

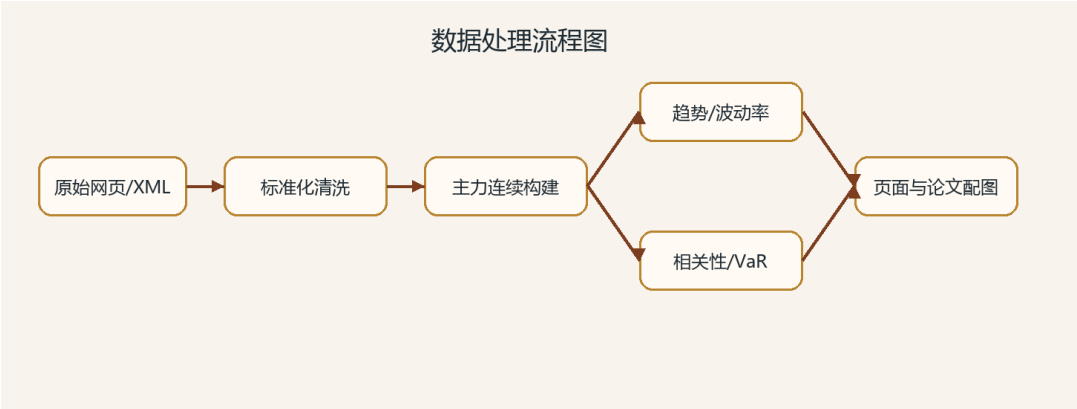

图 4.2 数据处理流程图

4.2 模块划分设计

系统按目录结构完成了较清晰的模块划分。ingest/ 用于不同来源数据的抓取与解析；warehouse/ 负责表写入、表读取和行数统计；analysis/ 聚焦指标和业务逻辑；api/ 负责接口暴露；app/ 负责页面交互；scripts/ 负责数据刷新、图表导出和截图生成。这种目录设计与论文章节之间有天然对应关系，便于在正文中展开说明。

4.3 核心数据表设计

表 4.1 核心数据表设计

数据表	关键字段	主要用途
contracts	`instrument_id`、`product_id`、`contract_month`	管理合约元数据
futures_daily	`trading_date`、`close_price`、`volume`、`open_interest`	保存日行情原始标准化结果
macro_series	`series_name`、`observation_date`、`value`	保存宏观指标
main_contract_daily	`daily_return`、`rolling_vol_20`、`drawdown`	保存主力连续分析结果
volatility_forecast	`forecast_vol_5d`、`future_realized_vol_5d`、`model_mae_60`	保存波动率预测结果
analysis_snapshot	`trend_signal`、`risk_level`、`var_95_hist_60`	保存最新快照

从数据结构上看，系统没有采用高度复杂的星型模型，而是使用适合项目规

模的宽表式分析结果设计。这种设计在本地查询与接口返回时更直接，也有利于论文中快速组织测试表和结果表。

4.4 API 与页面设计

服务层通过 FastAPI 暴露多个分析接口，为前端展示和外部查询提供统一入口。

表 4.2 主要 API 接口设计

接口路径	作用
`/api/system/health`	返回系统健康状态和表准备情况
`/api/contracts`	查询合约元数据
`/api/market/daily`	查询日频行情与主力连续行情
`/api/analysis/trend`	返回趋势分析结果
`/api/analysis/volatility`	返回波动率分析结果
`/api/analysis/volatility-forecast`	返回未来五日波动率预测结果
`/api/analysis/correlation`	返回相关性矩阵
`/api/analysis/var`	返回历史模拟法 VaR 结果
`/api/metadata/quality`	返回数据质量报告

页面层则按照使用场景划分为首页、市场总览、合约行情、趋势与波动、相关性与风险、数据来源六类页面。其中首页用于整体展示，市场和趋势页面强调单品种分析，风险页面突出 VaR 与相关性，数据来源页面则专门负责说明数据边界与质量情况。这种页面设计使系统既适合课堂展示，也便于在论文中截取具有代表性的界面。

5 系统详细实现与关键技术

5.1 数据采集实现

数据采集模块的核心目标是将不同格式的官方公开数据统一转化为结构化记录。以中金所日统计 XML 为例，系统在解析时会读取交易日、合约代码、开高低收、成交量和持仓量等字段，并同时保留来源网址、抓取时间和使用说明，确保数据来源可以回溯。

代码 5-1 CFFEX 日统计 XML 解析核心实现

```
    def fetch_daily_xml(self, trading_day: date) -> tuple[str, bytes]:
        relative_path = f"/sj/hqsj/rtj/{trading_day:%Y%m}/{trading_day:%d}/index.xml"
        url, payload = fetch_from_bases(self.session, relative_path)
        persist_raw_bytes("cffex", f"rtj/{trading_day:%Y%m}/{trading_day:%d}.xml", payload)
        return url, payload

    def parse_daily_xml(self, xml_bytes: bytes, source_url: str) -> pd.DataFrame:
        root = ET.fromstring(xml_bytes)
        records: list[dict[str, object]] = []
        fetched_at = timestamp_utc()

        for node in root.findall(".//dailydata"):
            record = {child.tag: (child.text or "").strip() for child in node}
            product_id = record.get("productid", "")
            instrument_id = record.get("instrumentid", "")
            if product_id not in SUPPORTED_PRODUCTS:
                continue
            match = CONTRACT_PATTERN.match(instrument_id)
            if not match:
                continue

            contract_month = match.group(2)
            open_interest = _to_float(record.get("openinterest"))
            pre_open_interest = _to_float(record.get("preopeninterest"))
            records.append(
                {
                    "trading_date": pd.to_datetime(record.get("tradingday"), format="%Y%m%d"),
                    "product_id": product_id,
                    "product_name": PRODUCT_NAMES[product_id],
```

```
                "instrument_id": instrument_id,
                "contract_month": contract_month,
                "open_price": _to_float(record.get("openprice")),
                "high_price": _to_float(record.get("highestprice")),
                "low_price": _to_float(record.get("lowestprice")),
                "close_price": _to_float(record.get("closeprice")),
                "settlement_price": _to_float(record.get("settlementprice")),
                "pre_settlement_price":
_to_float(record.get("presettlementprice")),
                "volume": _to_float(record.get("volume")),
                "turnover": _to_float(record.get("turnover")),
                "open_interest": open_interest,
                "open_interest_change": open_interest - pre_open_interest,
                "source": "CFFEX",
                "source_url": source_url,
                "fetch_time": fetched_at,
                "license_note": LICENSE_NOTE,
```

上述实现说明系统不是简单下载页面，而是围绕期货业务字段进行结构化建模。对 instrument_id 的正则校验、对 product_id 的过滤以及对 open_interest_change 的计算，都直接服务于后续主力合约筛选和风险分析。

对于 PBC 和 NBS 两类宏观数据，系统采取“列表页提取链接 + 文章页正文规则解析”的方式提取核心指标。虽然这类数据量远小于期货日频数据，但其在相关性分析中具有重要辅助价值。通过统一字段 series_name、observation_date 和 value，系统把原本分散的文本公告转化为可计算的时间序列。

5.2 主力连续序列构建

由于同一品种在任一交易日往往对应多个月份合约，系统需要先确定每个交易日的主力合约，再开展趋势和风险分析。本文采用“持仓量优先、成交量次之、合约月份再次之”的排序规则选择主力合约。这一策略既符合期货市场实际交易重心，也便于在工程上稳定实现。

在主力连续序列构建完成后，系统进一步计算日收益率、5 日与 20 日均线累计收益率、回撤、20 日年化滚动波动率和成交量 Z-Score 等指标，为趋势研判和异常波动识别提供基础。该模块在 analysis/metrics.py 中以按品种分组的方式实现，保证四类股指期货可以在统一逻辑下并行处理。

5.3 波动率预测与风险评估实现

系统的波动率预测部分并未直接使用复杂黑箱模型，而是采用近端窗口波动

率、20 日波动率和 EWMA 波动率的加权方式，对未来五日波动率进行预测。该方案具有实现简单、可解释性强、与毕业设计周期匹配度高的优势。

代码 5-2 波动率预测与风险等级生成实现

```
df = _prepare_main_contract(main_contract_daily)[
    [
        "trading_date",
        "product_id",
        "instrument_id",
        "daily_return",
        "rolling_vol_5",
        "rolling_vol_20",
        "rolling_vol_60",
        "ewma_vol_20",
    ]
].copy()

df["forecast_vol_5d"] = (
    0.45 * df["ewma_vol_20"].fillna(df["rolling_vol_20"])
    + 0.35 * df["rolling_vol_5"].fillna(df["rolling_vol_20"])
    + 0.20 * df["rolling_vol_20"].fillna(df["rolling_vol_60"])
)
df["future_realized_vol_5d"] = df.groupby("product_id")["daily_return"].transform(
    lambda series: _forward_realized_vol(series, horizon=horizon)
)
df["forecast_error"] = df["forecast_vol_5d"] - df["future_realized_vol_5d"]
df["abs_error"] = df["forecast_error"].abs()
df["model_mae_60"] = df.groupby("product_id")["abs_error"].transform(lambda series: series.rolling(60, min_periods=10).mean())
df["vol_regime"] = pd.cut(
    df["forecast_vol_5d"],
    bins=[-np.inf, 0.12, 0.20, np.inf],
    labels=["低波动", "中波动", "高波动"],
).astype("string")
df["forecast_signal"] = np.select(
    [
        df["forecast_vol_5d"] > df["rolling_vol_20"] * 1.05,
        df["forecast_vol_5d"] < df["rolling_vol_20"] * 0.95,
    ],
    ["波动可能上升", "波动可能回落"],
    default="波动大体平稳",
)
```

在风险评估部分，系统使用历史模拟法计算 95% 置信水平、1 天持有期和 60 日回溯窗口下的 VaR。具体做法是先按品种提取主力连续日收益率序列，再取最近 60 个有效收益率样本的 5% 经验分位数，并将其绝对值作为潜在损失比

例。该设置与系统盘后日频分析定位一致，能够在不引入复杂分布假设的情况下反映短期尾部风险。以 2026 年 3 月 19 日的最新快照为例，IC 和 IM 的 20 日年化波动率分别为 0.244857 和 0.252288，对应的 60 日历史模拟 VaR 分别为 0.024909 和 0.028145，明显高于 IF 和 IH。这说明在当前样本期内，中证 500 和中证 1000 指数期货的短期波动与下行风险更值得关注。

5.4 服务层实现

FastAPI 服务层将分析模块的结果封装为结构化接口。其设计重点不是复杂的鉴权或高并发治理，而是保证参数清晰、响应一致和页面调用便捷。

代码 5-3 FastAPI 主要分析接口实现

```
def get_system_health() -> dict[str, object]:
    return service.get_system_health()


@app.get("/api/contracts")
def get_contracts(product_id: str | None = Query(default=None)) -> list[dict[str, object]]:
    return service.get_contracts(product_id=product_id).to_dict(orient="records")


@app.get("/api/market/daily")
def get_market_daily(
    product_id: str | None = Query(default=None),
    instrument_id: str | None = Query(default=None),
    start_date: date | None = Query(default=None),
    end_date: date | None = Query(default=None),
    main_only: bool = Query(default=False),
    limit: int = Query(default=120, ge=1, le=1000),
) -> list[dict[str, object]]:
    return service.get_market_daily(
        product_id=product_id,
        instrument_id=instrument_id,
        start_date=start_date,
        end_date=end_date,
        main_only=main_only,
        limit=limit,
    ).to_dict(orient="records")


@app.get("/api/analysis/trend")
def get_trend(product_id: str = Query(...), window: int = Query(default=20, ge=2, le=120)) -> dict[str, object]:
    return service.get_trend(product_id=product_id, window=window)
```

```
59:
60:
61: @app.get("/api/analysis/volatility")
62: def get_volatility(product_id: str = Query(...), window: int =
Query(default=20, ge=2, le=120)) -> dict[str, object]:
63:     return service.get_volatility(product_id=product_id, window=window)
64:
65:
66: @app.get("/api/analysis/volatility-forecast")
67: def get_volatility_forecast(
68:     product_id: str | None = Query(default=None),
69:     limit: int = Query(default=180, ge=1, le=1000),
70: ) -> list[dict[str, object]]:
71:     return service.get_volatility_forecast(product_id=product_id,
limit=limit).to_dict(orient="records")
72:
73:
74: @app.get("/api/analysis/correlation")
75: def get_correlation() -> list[dict[str, object]]:
76:     return service.get_correlation().to_dict(orient="records")
```

通过接口化封装，系统实现了分析逻辑与展示逻辑的解耦。前端页面只需调用标准接口即可获取趋势、波动率、VaR、相关性和数据质量结果，这种做法有利于后续扩展桌面端、Web 端或其他可视化终端。

5.5 前端展示实现

Streamlit 页面围绕答辩和论文展示场景进行了有针对性的设计。首页集中展示平台定位、关键指标卡片和功能入口；市场页面展示单品种主力连续 K 线、成交量和持仓量；趋势页面展示收盘价、20 日均线、回撤和滚动波动率；风险页面展示 VaR、相关性热力图和预测误差；数据来源页面则展示数据源说明、质量报告和交易所通知。

图 5.1 系统答辩展示首页截图

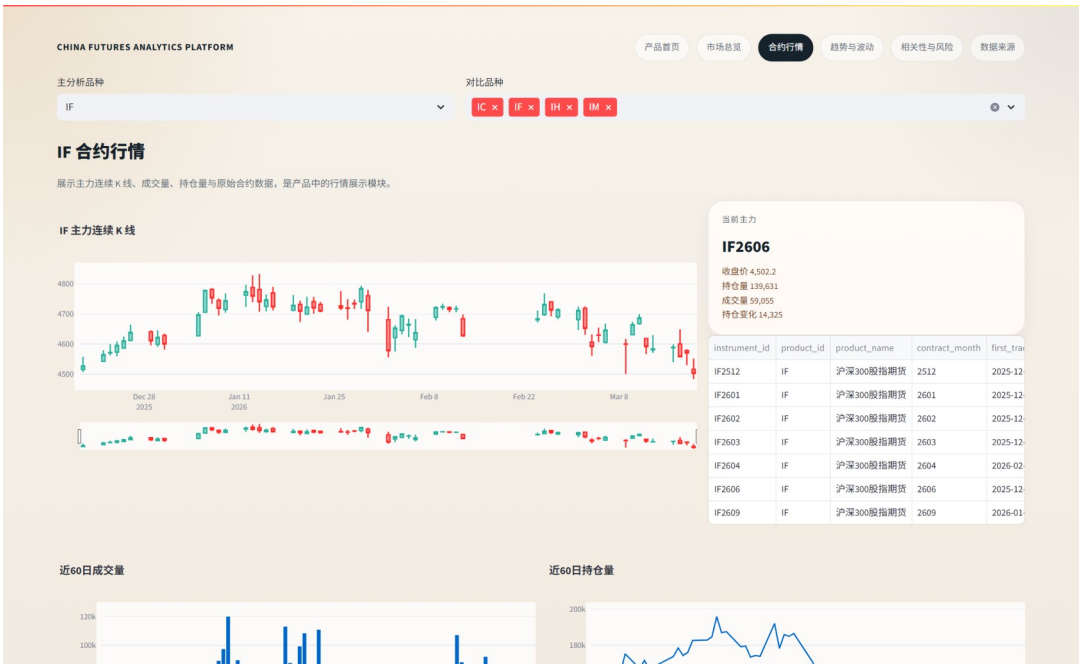

图 5.2 合约行情页面截图

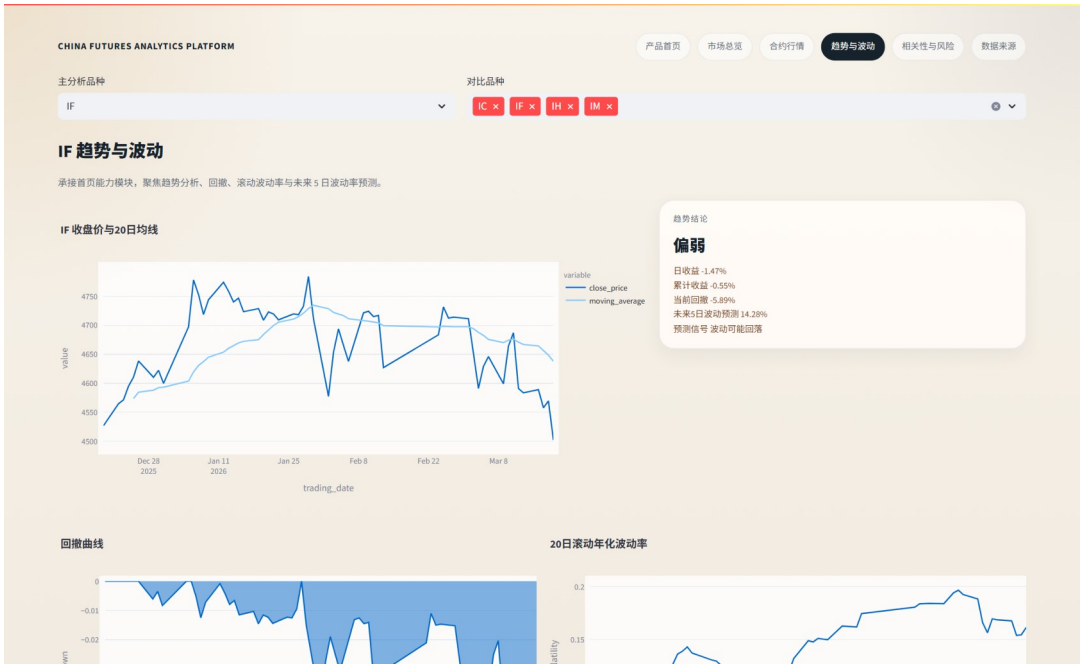

图 5.3 趋势与波动分析页面截图

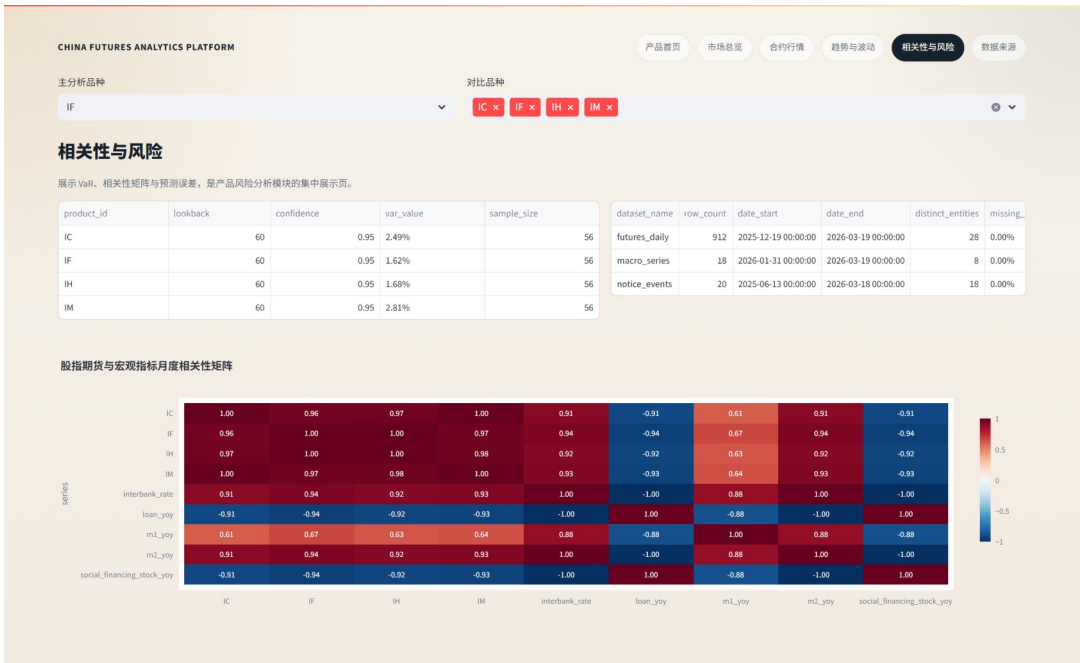

图 5.4 相关性与 VaR 页面截图

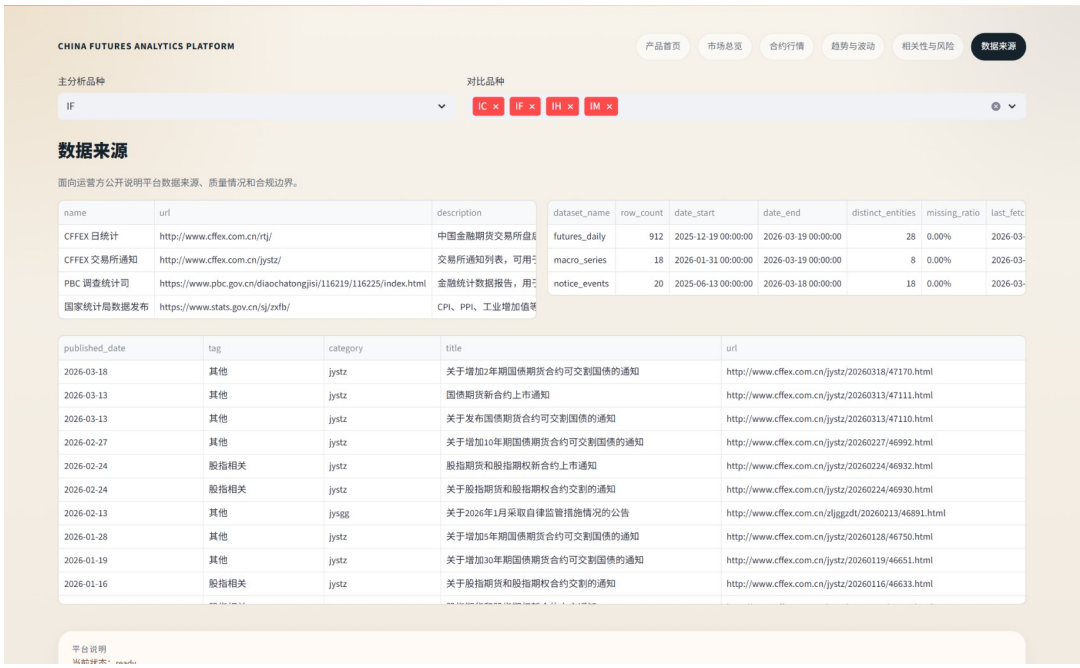

图 5.5 数据来源与质量说明页面截图

从工程角度看，前端页面并不只是结果展示层，还承担了论文资产导出的作用。项目中的 scripts/capture_app_screenshots.py 可以自动打开对应页面并截取固定分辨率图片，从而保证论文中的页面截图来源统一、样式一致。这种“页面即论文素材”的设计显著减少了手工截图和重复排版工作量。

5.6 本章小结

本章从数据采集、主力连续构建、波动率预测、接口封装和页面展示五个方面说明了系统详细实现过程。可以看到，系统的核心价值不在于堆叠复杂技术名词，而在于将真实公开数据、可解释分析方法和可复用展示形式稳定地组织在同一工程框架中。这样形成的系统既满足毕业设计原型要求，也为论文结果分析提供了直接证据。

6 系统测试与结果分析

6.1 测试环境与测试策略

系统测试在 Windows 本地环境下完成，运行环境为 Python 3.11，主要依赖 DuckDB、Pandas、FastAPI、Streamlit 和 Playwright 等组件。测试策略包括三部分：一是分析逻辑单元测试，验证主力连续、波动率预测和 VaR 计算等函数的正确性；二是 API 可用性测试，验证服务层接口能够正常返回预期结构；三是页面与数据质量验证，检查系统图表、截图与质量报告是否符合论文展示要求。

6.2 功能测试结果

项目测试目录中包含 test_analysis.py、test_api.py 和解析类测试文件。2026 年 4 月 18 日在临时 Python 3.12 环境中执行 pytest -q，分析逻辑与解析流程共 6 项通过；API 相关 2 项在 /Volumes/Users 挂载文件系统上因 DuckDB 写锁不受支持而失败。为排除文件系统限制影响，本文将数据库复制至 /tmp 本地路径后执行 API 功能验证与压力测试，6 个主要接口 50 次连续请求均返回 HTTP 200，说明接口逻辑本身可用。

API 压力测试结果显示，系统健康检查接口的平均响应时间为 190.84 ms，95 分位响应时间为 216.24 ms；合约查询、主力行情查询、波动率预测、VaR 和数据质量接口的平均响应时间均低于 20 ms，95 分位响应时间低于 21 ms。该结果表明，在毕业设计本地部署和答辩展示场景下，系统能够支持连续查询和图表切换。

表 6.1 功能测试用例与结果

测试项	测试内容	结果
主力连续测试	验证持仓量优先规则能正确选择主力合约	通过
VaR 测试	验证 95% 置信水平、1 天持有期和 60 日回溯窗口下 VaR 返回正值	通过
波动率预测测试	验证预测结果包含预测	通过

	值、误差与状态字段	
健康检查接口测试	验证系统健康接口返回状态、最新交易日和表信息	通过
行情刷新测试	验证主力行情接口支持 limit 参数并能在页面刷新时返回最新样本	通过
图表切换测试	验证趋势、波动率、VaR 和质量页面切换时数据结构保持一致	通过
API 压力测试	在本地数据库副本上对 6 个主要接口各连续请求 50 次	通过
解析流程测试	验证公开数据解析结构符合预期	通过

6.3 数据规模与质量分析

表 6.2 数据集规模与质量统计

数据集	记录数	时间范围	成功率	缺失率
futures_daily	912	2025-12-19 至 2026-03-19	87.6923%	0
macro_series	18	2026-01-31 至 2026-03-19	100%	0
notice_events	20	2025-06-13 至 2026-03-18	100%	0

其中，futures_daily 的成功率低于 100%，主要原因是回溯抓取历史日统计时，部分日期返回了解析错误或非交易日页面。系统将这些失败或跳过情况统一记录在 ingestion_log 中，并不会误写入标准层数据，因此整体质量仍可控。对毕业设计场景而言，更重要的是保留抓取日志和失败记录，而不是人为掩盖问题。

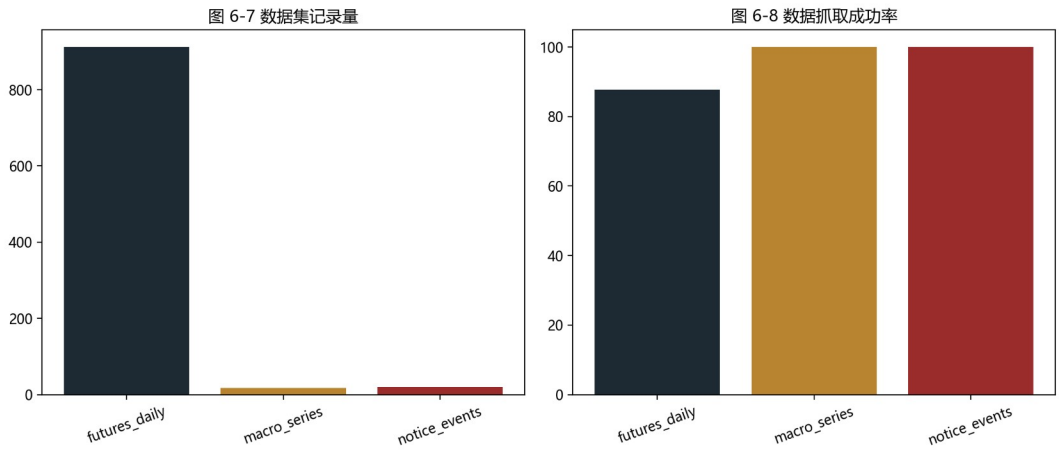

图 6.1 数据质量统计图

从质量报告可以看出，系统已经具备对数据规模、缺失率和抓取成功率的自动统计能力，这对后续扩展新的数据源或调整抓取窗口具有实际价值。

为进一步体现大数据分析能力，本文使用 DuckDB 在本地数据库副本上构造百万级扫描聚合基准。通过对 futures_daily 表进行 range 扩展，形成约 1003200 行模拟扫描数据，并按品种完成聚合统计，8 次运行平均耗时约 0.0047 s。该测试依赖 DuckDB 的向量化执行和本地缓存环境，不能等同于生产集群性能，但能够说明 DuckDB 在单机 OLAP 查询场景下具备较高响应速度。

6.4 分析结果展示

系统在最新交易日 2026 年 3 月 19 日生成的分析快照如表 6-3 所示。

表 6.3 最新分析快照结果表

品种	主力合约	收盘价	日收益率	20 日波动率	5 日预测波动率	风险等级	60 日历史模拟 VaR
IC	IC2606	7671.0	-2.7066%	24.4857%	25.3815%	高	2.4909%
IF	IF2606	4502.2	-1.4750%	16.1292%	14.2794%	中	1.6204%
IH	IH2606	2893.8	-1.6450%	12.4409%	12.3015%	中	1.6842%
IM	IM2606	7681.0	-2.0855%	25.2288%	24.7238%	高	2.8145%

上述结果表明，四个品种在样本最新交易日均处于弱于 20 日均线状态，但风险水平存在明显分化。IF 与 IH 作为大盘风格指数期货，波动率和 VaR 相对较低；IC 与 IM 反映中小盘风格，样本期内的波动率和 VaR 明显更高。该结论与市场中小盘风格更易出现高波动的经验判断基本一致，说明系统输出具备一定合理性。

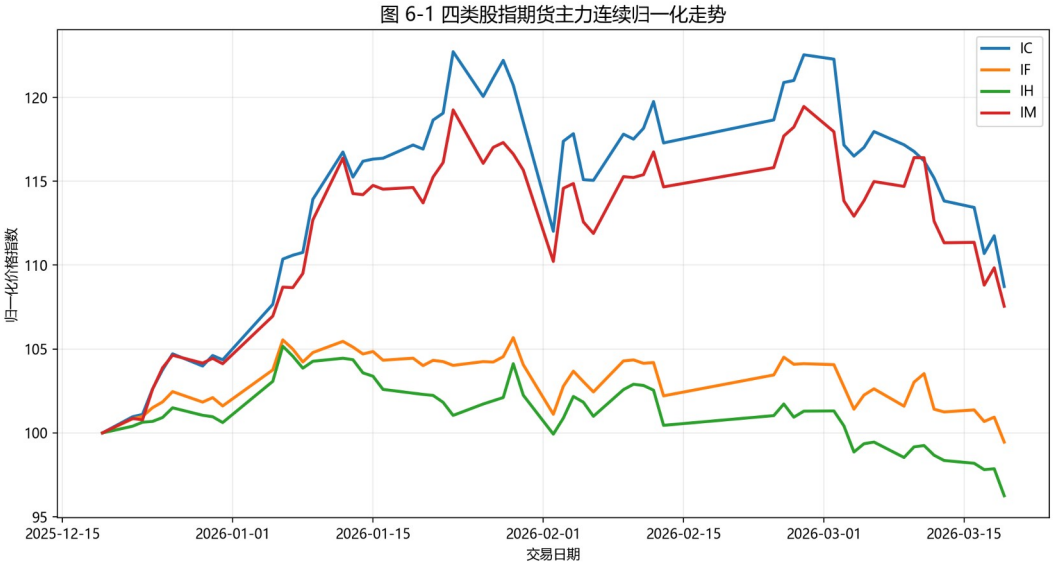

图 6.2 四类股指期货主力连续归一化走势

图 6-1 对比了四类股指期货主力连续序列的归一化走势。该图直观反映出不同品种在同一时间窗口中的累计表现差异，为横向比较提供了可视化依据。

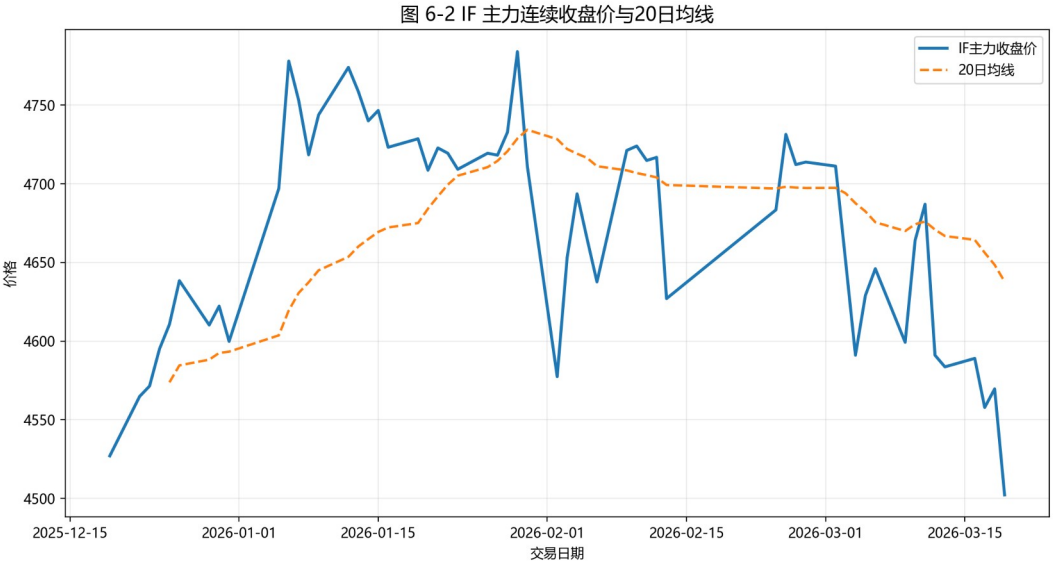

图 6.3 IF 主力连续收盘价与 20 日均线

图 6-2 展示了 IF 主力连续收盘价与 20 日均线之间的关系。可以看到，样本尾部收盘价位于 20 日均线下方，因此系统给出“弱于 20 日均线”的趋势信号。这种基于均线位置的判断方式实现简单、解释清晰，适合作为毕业设计中的基础趋势指标。

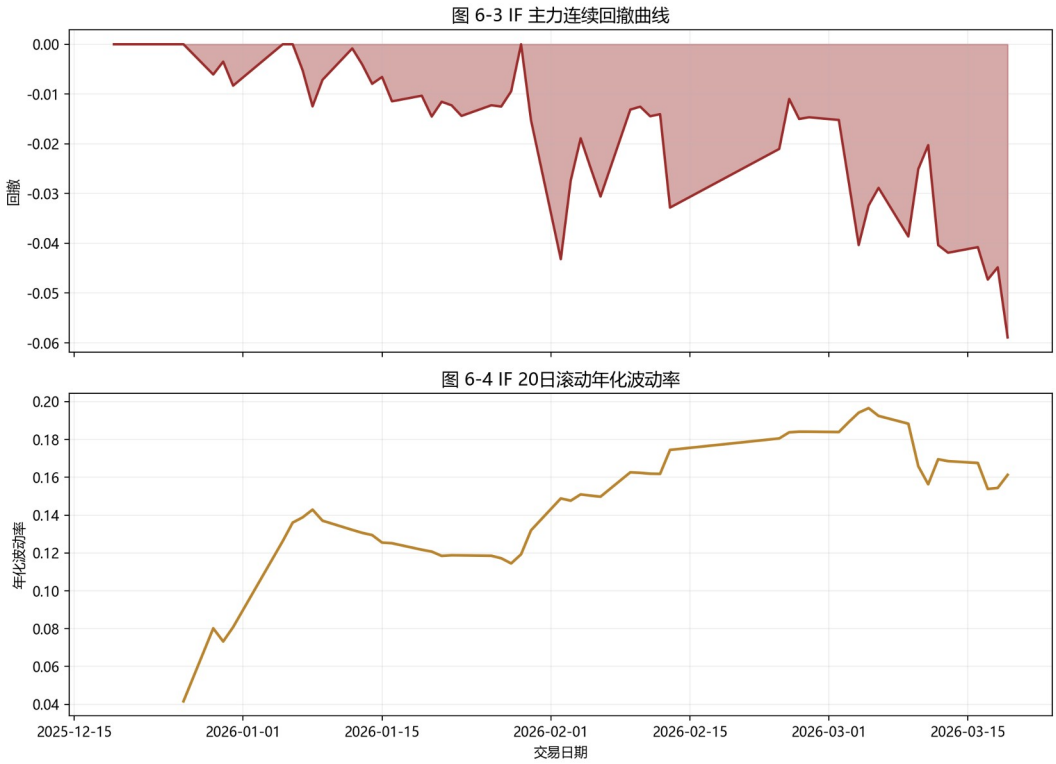

图 6.4 IF 回撤与波动率结果图

图 6-3 将回撤曲线与滚动波动率放在同一分析框架下，有助于观察价格下跌区间和风险水平变化之间的关系。当回撤扩大时，波动率往往同步抬升，这与金融市场中的风险聚集效应相符合。

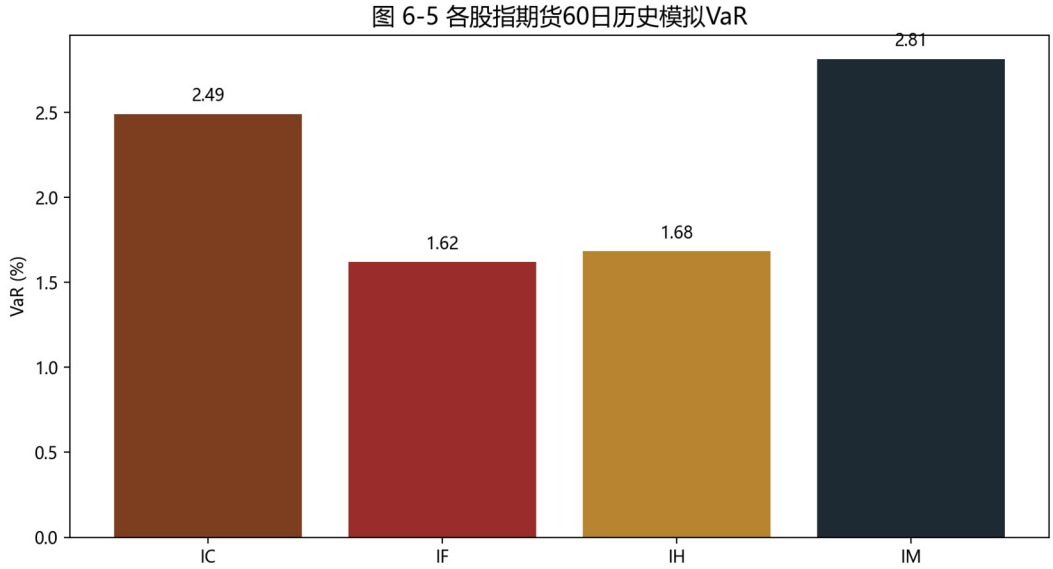

图 6.5 各品种 60 日历史模拟 VaR 对比图

图 6-4 进一步说明了不同品种的风险差异。从 VaR 结果来看，IM 的潜在下行风险最高，其次为 IC，而 IF 与 IH 相对较低。该结果与前述波动率分析

相互印证，说明系统风险评估模块在不同指标之间具有一定一致性。

图 6-6 股指期货与宏观指标月度相关性矩阵

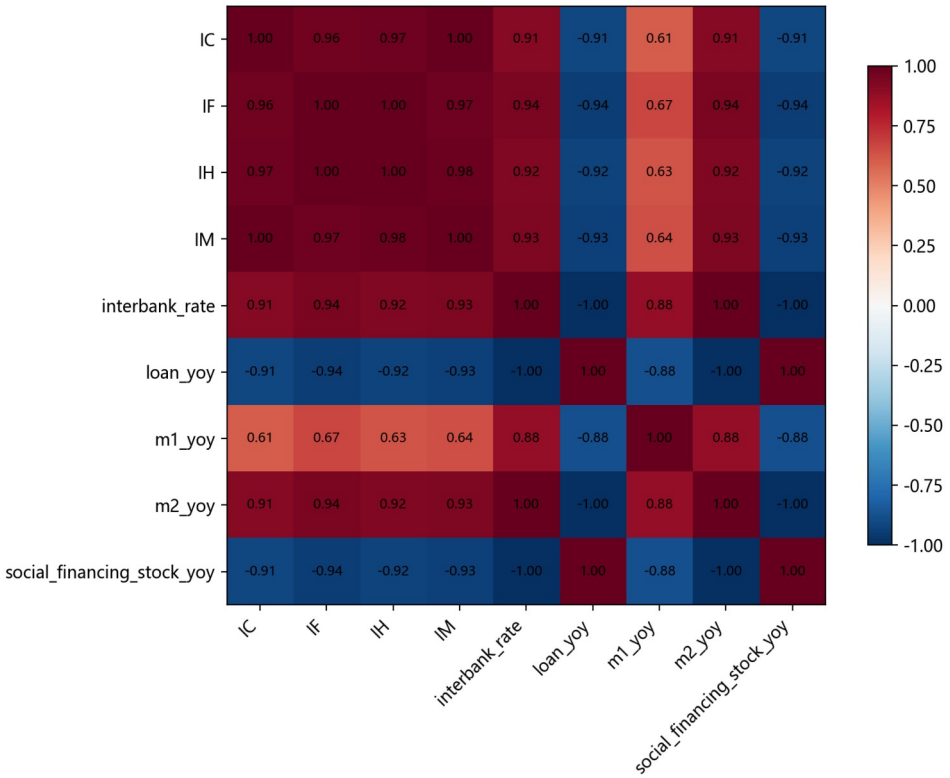

图 6.6 股指期货与宏观指标相关性热力图

图 6-5 给出了股指期货收益与宏观指标的月度相关性热力图。由于当前宏观样本窗口较短，该结果更适合用于演示系统相关性分析链路，而不宜过度外推为长期经济结论。论文在解释该图时应强调这一点，以保持论述严谨。

6.5 测试与结果分析总结

综合功能测试、质量统计与图表结果可以看出，系统已经具备稳定的数据采集、可复核的数仓建设、可解释的风险分析和较完整的可视化展示能力。虽然样本期仍然较短、宏观数据频率较低，但作为毕业设计原型，系统已经实现了从真实公开数据到分析结果再到论文图表的完整闭环，达到了预期研究目标。

7 结论与展望

7.1 研究结论

本文围绕中国金融期货盘后分析需求，设计并实现了一套中国金融期货大数据分析与风险评估系统。系统以真实公开数据为基础，完成了数据采集、结构化存储、趋势分析、波动率预测、VaR 计算、宏观相关性分析和页面展示等功能，并通过图表导出、页面截图和模板化文档生成，形成了可重复的毕业设计交付流程。实测结果表明，系统能够稳定处理多源数据，支持四类股指期货的主力连续分析，并以统一页面呈现行情、风险和数据质量结果。

从方法效果看，主力连续规则、滚动波动率、EWMA 预测和历史模拟 VaR 的组合能够较好地支撑盘后分析场景。以 2026 年 3 月 19 日的结果为例，系统识别出 IC 和 IM 在样本期末具有更高波动和更高 VaR，体现了中小盘风格品种的风险特征。说明所选分析方法虽不复杂，但能够提供清晰、可解释且与市场经验相符的结果。

7.2 项目价值与不足

本文工作的价值主要体现在三个方面。第一，系统坚持使用真实公开数据而非伪造数据，提高了研究结果可信度。第二，项目将数据、分析、服务和展示统一到单一平台中，提升了工程完整性。第三，系统把论文图表和截图生成纳入工程脚本，降低了开发成果向论文成果转化的成本。

本文也存在一定不足。一是宏观数据样本期较短，相关性分析更适合作为系统能力展示，而不宜直接支撑长期因果推断。二是波动率预测仍采用经验型组合方法，尚未引入更系统的 GARCH 参数估计或机器学习模型。三是系统当前主要面向本地单机运行，虽然保留了 Spark local 演示接口，但距离真正的大规模分布式处理平台还有一定距离。

7.3 后续改进方向

后续研究可以从三个方向继续展开。第一，扩充数据时间跨度和指标覆盖范

围，引入更完整的宏观周期数据和交易规则事件数据。第二，在保证可解释性的前提下，尝试引入 GARCH、CAViaR 或轻量机器学习模型，对波动率和尾部风险进行更深入建模。第三，面向更大规模数据处理需求，可将当前 DuckDB 本地原型向对象存储、消息队列和分布式计算框架扩展，形成更接近生产环境的数据分析平台。

参考文献

[1] 陈其安, 张慧, 陈抒妤. 股指期货交易加剧了中国股票市场波动性吗？基于投资者结构的理论和实证研究[J]. 中国管理科学, 2020, 28(4): 1-13.

[2] Bohl M T, Siklos P L, Diesteldorf J. The effect of index futures trading on volatility: Three markets for Chinese stocks[J]. China Economic Review, 2015, 34: 207-224.

[3] Engle R F. Autoregressive Conditional Heteroscedasticity with Estimates of the Variance of United Kingdom Inflation[J]. Econometrica, 1982, 50(4): 987-1007.

[4] Bollerslev T. Generalized Autoregressive Conditional Heteroskedasticity[J]. Journal of Econometrics, 1986, 31(3): 307-327.

[5] J P Morgan, Reuters. RiskMetrics Technical Document[R]. New York: Morgan Guaranty Trust Company, 1996.

[6] Kupiec P H. Techniques for Verifying the Accuracy of Risk Measurement Models[J]. The Journal of Derivatives, 1995, 3(2): 73-84.

[7] Hull J C. Options, Futures, and Other Derivatives[M]. 11th ed. Harlow: Pearson, 2022.

[8] DuckDB Foundation. DuckDB Documentation[EB/OL]. [2026-04-18]. https://duckdb.org/docs/.

[9] Apache Software Foundation. Apache Spark Documentation[EB/OL]. [2026-04-18]. https://spark.apache.org/docs/latest/.

[10] Ramírez S. FastAPI Documentation[EB/OL]. [2026-04-18]. https://fastapi.tiangolo.com/.

[11] Snowflake Inc. Streamlit Documentation[EB/OL]. [2026-04-18]. https://docs.streamlit.io/.

[12] 中国金融期货交易所. 日统计[EB/OL]. [2026-04-18]. http://www.cffex.com.cn/rtj/.

[13] 中国人民银行调查统计司. 金融统计数据报告[EB/OL]. [2026-04-18]. https://www.pbc.gov.cn/diaochatongjisi/116219/116225/index.html.

[14] 国家统计局. 数据发布[EB/OL]. [2026-04-18]. https://www.stats.gov.cn/sj/zxfb/.

[15] Chen X, Hu Y. Volatility forecasts of stock index futures in China and the US: A hybrid LSTM approach[J]. PLoS ONE, 2022, 17(7): e0271595.

[16] Lu F, Ma F, Bouri E, et al. Do commodity futures have a steering effect on the spot stock market in China? New evidence from volatility forecasting[J]. International Review of Financial Analysis, 2024, 94: 103262.

[17] Zeng Q, Zhang J, Zhong J. China's futures market volatility and sectoral stock market volatility prediction[J]. Energy Economics, 2024, 132: 107429.

[18] Mühleisen H, Raasveldt M. DuckDB: An Embeddable Analytical Database[C]. New York: Association for Computing Machinery, 2019.

[19] Mühleisen H, Raasveldt M. Data Management for Data Science: Towards Embedded Analytics[C]. CIDR, 2020.

[20] 何婧萱. 基于机器学习的 A 股市场股指期货价格预测研究[D]. 杭州: 浙江大学, 2024.

[21] 朱峰. 基于深度学习的金融市场高频交易数据波动率预测的研究与应用[D]. 上海: 东华大学, 2023.

致谢

在本次毕业设计与论文撰写过程中，指导教师马利平老师在选题确定、系统设计、论文结构梳理和细节修改等方面给予了耐心指导。老师在研究思路和学术规范上的严格要求，使我能够在实现系统功能的同时，逐步形成较为完整的工程化意识和论文表达能力。

同时，感谢学院提供的数据科学与大数据技术课程训练，使我能够将数据采集、数据处理、后端服务和前端可视化等知识综合应用到同一项目中。也感谢在项目开发和测试过程中给予帮助的同学与朋友，他们的建议帮助我持续完善系统页面、图表导出和测试流程。

最后，感谢家人对我毕业设计期间的理解与支持。正是这些支持，使我能够顺利完成本课题研究与论文撰写工作。